PART 1 — Concepts: Architecture, Enterprise, EA

1.1 What is Architecture?

Two definitions exist in the slides, and they complement each other:

Definition	What it says
ISO/IEC 42010:2007	A formal description of a system, or a detailed plan at component level, to guide its implementation.
Structural Definition	The structure of components, their interrelationships, and the principles and guidelines governing their design and evolution over time.

> **Plain English:** "Architecture" is the blueprint — the detailed plan — that tells you what pieces a system has, how they connect, and the rules for how the system should grow and change over time.

> **MEMORY HOOK —** *"Architecture = Blueprint + Rules."* A blueprint shows what is built. The rules explain how to build and how to change it. Architecture does both.

1.2 What is an Enterprise?

Two definitions are given:

- **Minoli, 2008:** "Any collection of corporate or institutional task-supporting functional entities that have a set of common goals or a single mandate."
- **ISO 2000:** "One or more organizations sharing a definite mission, goals, and objects to offer an output such as a product or service."

> **Plain English:** An enterprise is simply any organization — big or small, a hospital, a government agency, a startup — that has a shared goal and produces something. It can even be multiple organizations working together toward one mission.

> **EXAMPLE:** A university is an enterprise. Its mission is education. It has departments (HR, Finance, Academics, IT) that are separate but all work toward the same goal. A hospital system with multiple branches is also a single enterprise.

> **MEMORY HOOK —** *"Enterprise = Any org with a mission."* If it has a goal and produces output, it is an enterprise.

1.3 What is Enterprise Architecture (EA)?

Official Definition (Gartner Group):

> *"Enterprise Architecture (EA) is the process of translating business vision and strategy into effective enterprise change by creating, communicating, and improving the key principles and models that describe the enterprise's future state and enable its evolution."*

> **Plain English:** EA is the master plan for the entire enterprise. It takes the company's big-picture goals ("we want to be digital-first by 2026") and turns them into a structured, documented plan that covers the business operations, IT systems, data, and technology needed to get there.

EA acts like a master plan and takes into account:

- **Business strategy, goals, vision** and principles
- **Business operations**
- **Automation of processes**
- **Data processing**
- **Technological infrastructure**
- **Communication among different stakeholders** — promotes a common glossary so everyone speaks the same language

> **MEMORY HOOK — THE HOUSE ANALOGY (from the slides):** Think of your enterprise as a house. IT is not just a tool — it is the FRAME of the house. Without the frame, the house collapses. EA is the architect's blueprint BEFORE construction begins. You would never build a house without a plan. Likewise, you should never transform an enterprise without an EA.

> **EXAMPLE:** A bank wants to offer mobile banking. EA is the plan that says: here is what the mobile app needs, here is how it connects to the core banking system, here is what data it uses, here is what security standards apply, and here is how it aligns with the bank's goal of serving customers digitally.

1.4 Why Do We Need Enterprise Architecture?

Primary goal of EA: To **align IT-related activities with the goals of the enterprise.**

This includes managing:

- **All investments** in IT
- **Resources** — hardware, software, facilities, people
- **Strategic planning** of the EA's evolution over time
- **Up-to-date record** of the current ("as-is") situation

Two viewpoints on EA (from IBM):

Role	Focus Area
Solution Architect	Creates an IT-based solution for a specific business problem. Narrower focus.
Enterprise Architect	Has a broader, strategic view of the entire enterprise and how IT should be used across the whole organization.

> **MEMORY HOOK:** *"Solution Architect = Solves one room's problem. Enterprise Architect = Plans the whole house."*

1.5 The Four EA Domains (Layers)

Because EA is complex, it is broken into layers — called **architectural domains**. Each domain addresses a different part of the enterprise.

#	Domain	What It Covers	Think of it as...
1	Business Architecture	Business strategy, governance, organization, and key business processes.	The "What we do and how we operate" layer
2	Data Architecture	Structure of logical and physical data assets and data management resources.	The "What information we keep and how" layer
3	Application Architecture	Blueprint for individual applications, their interactions and relationships.	The "What software we use and how it talks" layer
4	Technology Architecture	Logical software and hardware capabilities.	The "What servers, networks, and tools run it all" layer

> **MEMORY HOOK — "BDAT"** (Business, Data, Application, Technology). Remember it as: *"Big Dogs Are Tall."* From widest/strategic (Business) down to most technical (Technology).

1.6 The Four Types of EA Stakeholders

In EA, different people care about different things — they have different **viewpoints**. For each stakeholder, we describe their role, their concerns, and respond in a language suitable for them (called **EAPs — Enterprise Architecture Patterns**).

#	Stakeholder	Their Main Concern	Plain English Role
1	Enterprise Architect	Consistency of overall architecture + strategic direction aligned with business goals.	The big-picture designer who makes sure all parts fit together.
2	Project Stakeholders (Project Sponsor, Solution Architects, Business Analysts, Project Leaders)	Building or changing a specific ICT system.	The hands-on builders working on specific projects.
3	CIO / Strategic Planner	Strategic planning and decision-making on how ICT best supports business goals.	The executive who decides where IT money goes and why.
4	Internal Auditor / Controller	Controlling how efficiently ICT is used; comparing EAPs to the actual ICT landscape.	The checker who makes sure the plan matches reality.

PART 2 — Key Definitions: Plain-English Glossary

Architecture

- **Official:** *The fundamental concepts or properties of a system in its environment embodied in its elements, relationships, and in the principles of its design and evolution.*

> **Plain English:** The structure, rules, and relationships of a system — the "how it works and how it grows" document.

Architecture Framework

- **Official:** *A conceptual structure used to plan, develop, implement, govern, and sustain an architecture.*

> **Plain English:** A toolkit or method — a ready-made approach you can follow to build your EA. Think of it like a cooking recipe for building enterprise architecture.

Architecture Model

- **Official:** *A representation of a subject of interest.*

> **Plain English:** A drawing, diagram, or description that shows what something looks like or how it works.

Information Technology (IT)

- **Official:** *The lifecycle management of information and related technology used by an organization.*

> **Plain English:** Everything involved in handling data with technology — from buying the hardware to maintaining the software to decommissioning old systems.

Modelling

- **Official:** *A technique through construction of models which enables a subject to be represented in a form that enables reasoning, insight, and clarity.*

> **Plain English:** Drawing pictures (diagrams) of your system so everyone can understand it, reason about it, and talk about it clearly.

Baseline Architecture

- **Official:** *The "as-is" or "current" architecture; includes architecture assets from all architecture domains.*

> **Plain English:** A snapshot of your enterprise RIGHT NOW — what systems, processes, and technology you currently have. The starting point.

Target Architecture

- **Official:** *Architecture which provides the details of a future state of an architecture being developed for an organization.*

> **Plain English:** Where you WANT to be — the future blueprint. Your destination.

TAFIM

- **Official:** *Technical Architecture Framework for Information Management — a reference model for enterprise architecture by and for the United States Department of Defense.*

> **Plain English:** An early EA framework created by the US military to manage their huge, complex IT systems.

Enterprise Continuum (TOGAF)

- **Official:** *Explains how certain generic solutions can be customized and used as per specific requirements of an organization. Enables organization of re-usable architecture artifacts and solution assets.*

> **Plain English:** A library of reusable EA solutions — from generic templates to company-specific implementations. Instead of starting from zero, you reuse what works.

Architecture Repository (TOGAF)

- **Official:** *Used to store diverse types of architectural outputs, each at varying levels of abstraction, created by ADM.*

> **Plain English:** The filing cabinet for all your EA documents, diagrams, and outputs. Allows architects at all levels to access and collaborate on shared work.

> **MEMORY HOOK — Baseline vs. Target:** Baseline = "Where am I NOW?" (GPS current location). Target = "Where am I GOING?" (GPS destination). EA is the route planning in between.

PART 3 — TOGAF Standard and the Evolution of EE

3.1 What is TOGAF?

TOGAF = **The Open Group Architecture Framework**. It is a freely available, widely used framework for building an Enterprise Architecture.

> **Plain English:** TOGAF is like an instruction manual for building EA. Instead of figuring out how to do it from scratch, organizations follow TOGAF's proven steps, tools, and templates. It is the most widely used EA framework in the world.

> **MEMORY HOOK:** *"TOGAF = The Official Guide to Architecture Framework."* It gives you a tested, repeatable process so you do not reinvent the wheel.

3.2 TOGAF ADM — The Architecture Development Method

ADM is the core of TOGAF — a **tested and repeatable process for developing architectures**. It has **nine phases**, running from the **Preliminary Phase** all the way to **Architecture Change Management**.

> **MEMORY HOOK:** *"ADM = The Step-by-Step Recipe."* TOGAF gives you 9 ordered steps to build your EA. Like baking — you do not skip steps or do them out of order.

3.3 TOGAF Architecture Content Framework (ACF) — 3 Work Product Types

TOGAF uses a framework to organize all the work produced. There are three categories:

Type	Official Definition	Plain English	Example
Deliverable	A work product formally reviewed and agreed upon by stakeholders. Typical output of projects, usually in document form.	A final output that everyone has signed off on.	An Architecture Vision document approved by the board
Artifact	A work product that describes some aspect of an architecture.	A diagram, list, or matrix that shows part of the architecture.	A use-case diagram, requirements catalog, business interaction matrix
Building Block	A component of enterprise capability that can be combined with other building blocks to deliver architectures and solutions.	A reusable piece that fits together with other pieces to build capability.	A "Customer Authentication" component used across multiple apps

Two types of Building Blocks:

- **Architecture Building Blocks (ABBs)** — Describe the **capability expected from the architecture** (the specification/requirement level)
- **Solutions Building Blocks (SBBs)** — The actual **components used in the implementation** (the real-world solution level)

> **MEMORY HOOK:** *"ABB = What you need. SBB = What you build."* ABBs define the requirements; SBBs are the actual Lego bricks that satisfy those requirements.

3.4 The Evolution of EA — BSP to Modern EA

Enterprise Architecture has evolved through three key stages:

Aspect	BSP (1960s–1980s)	Early EA (1980s–1990s)	Modern EA (1990s–present)
Definitive Source	BSP (1975)	Spewak and Hill (1992)	TOGAF (2011)
Domains Covered	Organization, processes, data and information systems	Business, data, applications, and technology	Business, data, applications, and technology
Modeling Tools	Relationship matrices, info systems networks, flowcharts	Lists, relationship matrices, diagrams	Catalogs, matrices, and diagrams
Methodology	Describe current and desired states, prepare action plan, implement it	Describe current and future states, prepare implementation plan, implement it	Describe baseline and target states, prepare transition plan, implement, and REPEAT
Key Difference	N/A — Starting point	Pays more attention to technical aspects	Iterative in nature — keeps improving in cycles

> **MEMORY HOOK:** *"EA evolution = Plan → Technical Plan → Repeating Cycle."* Each generation got smarter: BSP planned once, Early EA added tech detail, Modern EA keeps repeating and improving.

PART 4 — ArchiMate® Modelling Language

4.1 What is ArchiMate®?

- **ArchiMate®** is an **open and independent modeling language for Enterprise Architecture**
- It provides instruments to **describe, analyze, and visualize relationships among business domains**
- It provides a **uniform representation for diagrams** that describe Enterprise Architectures

> **Plain English:** ArchiMate is the visual language used to draw EA diagrams. Just as architects use symbols on blueprints (a door symbol, a window symbol), EA architects use ArchiMate symbols to draw enterprise systems. It is standardized, so everyone reads the diagram the same way.

Relationship to TOGAF:

- ArchiMate produces most of the visual models for TOGAF: Business, Application/Data, and Technology domains
- If TOGAF is the *method*, then ArchiMate is the *drawing tool* you use within that method

> **MEMORY HOOK:** *"TOGAF = How to build. ArchiMate = How to draw it."*

4.2 ArchiMate® Structure — Models, Elements, Relationships

Everything in ArchiMate follows this hierarchy:

Term	Plain English Meaning
Model	A collection of concepts. Contains elements and relationships.
Element	A building block of the model. Can be a behavior, structure, motivation, or composite element.
Relationship	A connection between elements — shows how they interact or depend on each other.
Behavior Element	Something that represents an action, process, or function.
Structure Element	Something that represents a person, system, or device (the performer).
Motivation Element	Represents goals, principles, and drivers behind decisions.
Composite Element	A grouping element that contains other elements.
Relationship Connector	A special line/arrow type that connects elements with defined meaning.

4.3 ArchiMate® Layers and Aspects — The Big Picture

ArchiMate has **three main layers** (plus extended layers in the full framework):

Layer	What it Models	Main Active Element	Plain English
Business Layer	Business strategy, organization, processes, information needs	Business Actor, Business Role	"What the business does"
Application Layer	Information systems and applications that support the business	Application Component	"What software runs the business"
Technology Layer	IT infrastructure: servers, networks, hardware, system software	Node	"What hardware/network makes it run"

Each layer has three **Aspects**:

- **Active Structure** — Who/what PERFORMS the behavior (actors, systems, nodes)
- **Behavior** — The ACTIONS, processes, and functions being performed
- **Passive Structure** — The THINGS being acted on (data, materials, objects)

> **MEMORY HOOK — "Who Does What to What?":** Active Structure = WHO. Behavior = DOES. Passive Structure = WHAT.

4.4 Key ArchiMate® Element Definitions — Quick Reference

These definitions come directly from the slides. Bold terms are the vocabulary you need to know:

Business Layer Elements

Element	Official Definition	One-liner Memory
Business Actor	A business entity that is capable of performing behavior.	Any person or organization that "does" something.
Business Role	The responsibility for performing specific behavior, to which an actor can be assigned.	The "job title" or hat an actor wears.
Business Collaboration	An aggregate of two or more business active structure elements working together to perform collective behavior.	A team effort between actors.
Business Interface	A point of access where a business service is made available to the environment.	The front door of a business service.

Element	Official Definition	One-liner Memory
Business Process	A sequence of business behaviors that achieves a specific outcome.	Step-by-step workflow to get something done.
Business Function	A collection of business behavior based on chosen criteria, closely aligned to an organization.	A department's ongoing capability (e.g., "Billing").
Business Interaction	A unit of collective business behavior performed by two or more business roles.	When two roles work together on one thing.
Business Event	A business behavior element that denotes an organizational state change.	A trigger — something that happened (order placed, payment received).
Business Service	An explicitly defined exposed business behavior.	A service the business offers to others (e.g., "Loan Application Processing").

Application Layer Elements

Element	Official Definition	One-liner Memory
Application Component	An encapsulation of application functionality, modular and replaceable; exposes services through interfaces.	A self-contained software module (e.g., a login system).
Application Collaboration	An aggregate of two or more application components working together to perform collective application behavior.	Two software modules teaming up.
Application Interface	A point of access where application services are made available to a user or another component.	The API or screen through which a system is used.
Application Function	Automated behavior that can be performed by an application component.	What the software can do automatically.
Application Process	A sequence of application behaviors that achieves a specific outcome.	An automated workflow inside software.
Application Event	An application behavior element that denotes a state change.	A software trigger — something changed in the system.
Application Service	An explicitly defined exposed application behavior.	A capability a software system offers to the outside world.
Data Object	Data structured for automated processing.	A piece of organized data the system works with.

Technology Layer Elements

Element	Official Definition	One-liner Memory
Node	A computational or physical resource that hosts, manipulates, or interacts with other resources.	A server, PC, or device in the network.
Device	A physical IT resource upon which system software and artifacts may be stored or deployed.	The physical machine (laptop, server).
System Software	Software that provides an environment for storing, executing, and using other software or data.	The OS and middleware (e.g., Windows Server, Linux).
Technology Collaboration	An aggregate of two or more nodes working together to perform collective technology behavior.	A server cluster or load balancer group.
Technology Interface	A point of access where technology services offered by a node can be accessed.	The network port or connection point.
Path	A link between two or more nodes through which they can exchange data or material.	The network cable or wireless link.
Communication Network	A set of structures that connects systems for transmission, routing, and reception of data.	The LAN, WAN, or internet connecting everything.
Technology Service	An explicitly defined exposed technology behavior.	A service the tech layer offers (e.g., file storage, computing power).
Artifact	A piece of data used or produced in a software development process or by deployment and operation of a system.	A deployable file — like a .jar, .exe, or config file.

PART 5 — System Integration Concepts

5.1 What is System Integration? — Plain English First

> **Plain English:** System integration is the process of making different systems — which were built separately — work together as one unified whole. It is like making sure all the instruments in an orchestra play in harmony, not just individually.

Official Definition (ISO/IEC 15288:2015):

> *"System integration consists of a process that iteratively combines implemented system elements to form complete or partial system configurations in order to build a product or service. It is used recursively for successive levels of the system hierarchy."*

The ultimate goal of system integration:

> **To ensure that the individual system elements function properly as a whole and satisfy the design properties or characteristics of the system.**

The system integration process applies to any kind of **product system, service system, and enterprise system**

> **MEMORY HOOK:** *"Integration = Teamwork for Systems."* A single system working alone is fine. But when you need many systems to work as one — that is integration.

5.2 Why Do We Integrate? — Three Clear Purposes

The purpose of system integration can be summarized as:

1. **Verify Compatibility:** Completely assemble the implemented elements to make sure they are **compatible with each other**.
1. **Verify Performance:** Demonstrate that the aggregates of implemented elements **perform the expected functions** and meet measures of performance and effectiveness.
1. **Detect Defects:** Detect defects and faults related to design and assembly by submitting the aggregates to focused **Verification and Validation (V&V)** actions.

> **EXAMPLE:** A bank builds a new mobile banking app. Integration means: (1) checking if the app connects properly to the core banking system, (2) testing that it processes transactions correctly, and (3) finding bugs before customers experience them.

5.3 The 6 Integration Scenarios — When to Use Each One

Integration is broad. The slides present **six common integration scenarios**. Below, each one is explained with a plain-English definition, a memory hook, and — most importantly — **when you would use it versus the others**.

Scenario 1: Information Portal

Aspect	Details
Official Definition	An information portal can access information from diverse systems, aggregate, and present them in a single view.
Plain English	A single screen (portal) that pulls data FROM many different systems and shows it all in one place. The data does not move — it is read-only access.
Memory Hook	"PORTAL = Window to many rooms." You look through one window and see everything inside multiple rooms, but you do not move the furniture.
Real-World Example	A manager's dashboard that shows sales figures (from the CRM), inventory levels (from the warehouse system), and HR headcount (from the HR system) all on one screen.
USE THIS WHEN...	You need VISIBILITY across multiple systems without changing the data. Users want to see information from different sources in one place. READ-ONLY access is sufficient.
DO NOT USE WHEN...	You need to update data across systems, or when data needs to stay synchronized and up-to-date in real time (use Data Replication for that).

Scenario 2: Data Replication

Aspect	Details
Official Definition	Many organizations define policies that ensure continuous synchronization and replication of data at regular intervals of time, so data stays up to date on all the systems.
Plain English	Automatically copying data from one system to another at set intervals, so all copies stay current. Unlike a portal (read-only view), replication actually COPIES the data into other systems.
Memory Hook	"REPLICATION = Photocopier on a schedule." Every hour, the machine copies the master document to every branch office.
Real-World Example	A company replicates its product catalog database every night from headquarters to all regional branch systems, so every branch has the latest product pricing by morning.

Aspect	Details
USE THIS WHEN...	Data must be available locally in multiple systems (e.g., for performance or offline access). You can tolerate slight delays (e.g., hourly sync). Data changes infrequently enough that scheduled sync is acceptable.
DO NOT USE WHEN...	Data changes in real-time and every system needs the LATEST version instantly. Addresses that change every minute cannot wait for hourly replication.

Scenario 3: Shared Business Function

Aspect	Details
Official Definition	If the same set of data is stored in multiple systems, it leads to redundancy of data. A shared business function can replace redundant data.
Plain English	Instead of storing the same data in multiple places (which creates inconsistency), you create ONE central function that all systems CALL to get or process that data. Eliminates duplication.
The Trade-off	Trade-off between REDUNDANT DATA and SHARED FUNCTIONS depends on: (1) Amount of control over the systems, and (2) Rate of change of the data (e.g., a customer address may be needed frequently but changes rarely).
Memory Hook	"SHARED FUNCTION = One cashier for the whole mall." Instead of every store having its own cashier, they all share one central payment desk.
Real-World Example	Both the Billing system and the Shipping system need customer addresses. Instead of storing the address in both (causing inconsistency when it changes), both systems CALL a shared "Customer Address Service" to retrieve it.
USE THIS WHEN...	The same data or logic is needed by multiple systems. Consistency is critical (e.g., a customer address must always be the same in all systems). Redundancy is causing data quality problems.
DO NOT USE WHEN...	The data changes very frequently AND systems cannot tolerate the latency of calling a remote function every time (in that case, local replication might be better).

Scenario 4: Service-Oriented Architecture (SOA)

Aspect	Details
Official Definition	Shared business functions are commonly referred to as services. A service is a well-defined function that is universally available to perform a specific operation, made available for use by systems acting as service consumers.
Plain English	A formal, organized version of shared business functions — but at enterprise scale. In SOA, you create many reusable "services" (like building blocks), publish them in a directory, and any system in the enterprise can discover and use them.
Two Key Aspects	(1) Service Discovery — all services listed in a centralized service directory. (2) Service Negotiation — each service describes its interface so other apps can set up a communication contract with it.
Enterprise Service Bus (ESB)	An important part of SOA. The ESB provides CONNECTIVITY between the sender and receiver components of the SOA in a LOOSELY COUPLED manner — meaning systems do not need to know each other directly; they just connect through the bus.
Memory Hook	"SOA = The App Store for Enterprise Systems." Each service is like an app. You browse the directory, find what you need, agree to the terms (negotiation), and use it.
Real-World Example	A "Payment Processing" service is built once, registered in the service directory, and used by the mobile app, the website, the in-store terminal, and the partner portal — all without rebuilding payment logic for each.
USE THIS WHEN...	You have MANY systems that need to share the SAME capabilities at enterprise scale. You want reusability and loose coupling. You need formal contracts between systems.
DO NOT USE WHEN...	You have a small number of simple integrations (overkill for 2 systems). Or when you need very fast, fine-grained individual service deployments (consider Microservices instead).

Scenario 5: Distributed Business Process Management

Aspect	Details
Official Definition	A single business function can be spread across several applications present in an enterprise. It is very important to ensure coordination between the various applications. This can be done by implementing a Business Process Management System.
Plain English	A single end-to-end business process (like "fulfill a customer order") involves MANY different systems. A Business Process Management System (BPMS) acts as the CONDUCTOR — telling each system when to do its part.

Aspect	Details
Memory Hook	"DISTRIBUTED BPM = Orchestra Conductor." Each musician (system) knows their part, but the conductor ensures everyone plays at the right time in the right sequence.
Real-World Example	A customer places an order. The Order Management System records it, then the Inventory System reserves stock, then the Warehouse System prepares the shipment, then the Shipping System dispatches it, then the Billing System charges the customer. A BPMS coordinates all five steps.
USE THIS WHEN...	A single business process SPANS multiple applications and needs coordination. The process has a defined sequence of steps across systems. You need visibility and tracking of the end-to-end process.
DO NOT USE WHEN...	The process lives within a single application (no need for coordination). Or when you just need data to be shared (use Shared Functions or Replication instead).

Scenario 6: Business-to-Business (B2B) Integration

Aspect	Details
Official Definition	Present-day enterprises need to interact with several components that are external to their ecosystem, such as customers, partners, and so on. These external elements need access to many applications that are part of the enterprise.
Plain English	Connecting YOUR enterprise systems to systems OUTSIDE your organization — suppliers, customers, partners, government agencies. The integration crosses organizational boundaries.
Memory Hook	"B2B = Building a Bridge to Another Island." Your enterprise is one island. Your partner is another. B2B integration builds the bridge between them.
Real-World Example	A retailer's procurement system connects directly to a supplier's inventory system. When stock drops below a threshold, the retailer's system automatically sends a purchase order to the supplier's system — no human needed.
USE THIS WHEN...	Integration crosses ORGANIZATIONAL BOUNDARIES — you are connecting to a partner, supplier, or customer's systems. External parties need controlled access to your internal systems.
DO NOT USE WHEN...	All the systems involved belong to your own organization (use the other five scenarios for internal integration).

5.4 THE INTEGRATION DECISION GUIDE — When to Use Which Scenario

Use this as a quick mental flowchart when you face an integration problem:

If your situation is...	Use this scenario	Key Reason
"I need users to SEE data from multiple systems in one screen (read-only)"	Information Portal	Aggregates views without moving data
"I need the SAME data to exist in multiple systems and stay in sync"	Data Replication	Periodically copies data to all systems
"Multiple systems need the SAME data/logic but storing it twice causes inconsistency"	Shared Business Function	One central source of truth for data or logic
"I want to build reusable capabilities that many systems can call at enterprise scale"	Service-Oriented Architecture (SOA)	Formal service directory + contracts (ESB)
"A single end-to-end process crosses MULTIPLE internal systems and needs coordination"	Distributed Business Process Management	BPMS coordinates sequence across systems
"I need to connect MY systems with an EXTERNAL organization's systems"	Business-to-Business (B2B) Integration	Integration crosses organizational boundaries

> **MASTER MEMORY HOOK — "I DRESS SB" (Information portal, Data replication, REcycled/Shared function, Service-oriented architecture, Sequence-managed/Distributed BPM, B2B):** "I Dress Shared Services, Distribute B2B." But more practically, remember: Portal=View, Replication=Copy, Shared Function=Common Logic, SOA=Reusable Services, Distributed BPM=Coordination, B2B=External Bridge.

PART 6 — SOA vs. Microservices

6.1 Quick Plain-English Summary of Each

	Description
Service-Oriented Architecture (SOA)	SOA is an enterprise-wide approach to integration where shared functions are exposed as SERVICES, published to a centralized service directory, and connected through an Enterprise Service Bus (ESB). All services within the enterprise can discover and use each other through this shared infrastructure. It is designed for LARGE-SCALE ENTERPRISE INTEGRATION.
Microservices	Microservices is an architectural style where an application is broken into VERY SMALL, INDEPENDENT services — each doing ONE thing, running its own process, and communicating via lightweight APIs (usually REST or HTTP). Each microservice can be deployed, updated, and scaled independently. It is designed for AGILE APPLICATION DEVELOPMENT.

> **MEMORY HOOK:** *"SOA = Enterprise-wide Highway System. Microservices = Uber/Grab — independent cars going their own routes."* SOA builds shared infrastructure for the whole enterprise. Microservices are small, independent units that serve their own purpose.

6.2 Detailed Compare & Contrast Table

Aspect	SOA	Microservices
Scope	Enterprise-wide. Integrates MANY different applications and systems across the whole organization.	Application-level. Breaks a single application into small independent services.
Service Size	Services can be large — a "Loan Processing" service may handle many related functions.	Services are VERY small and fine-grained — each does exactly ONE thing (e.g., "Calculate Interest Rate Only").
Communication	Typically uses an Enterprise Service Bus (ESB) — a central middleware that routes messages between services.	Services communicate directly via lightweight APIs (REST, HTTP) — no centralized bus needed.
Data Management	Services often SHARE a central database or common data model.	Each microservice owns its OWN database. No shared data stores.

Aspect	SOA	Microservices
Coupling	Loosely coupled through the ESB — but the ESB itself can become a single point of failure or bottleneck.	Very loosely coupled — services are fully independent and do not depend on a central bus.
Deployment	Services are deployed together or in coordinated releases. Harder to update just one service without affecting others.	Each service can be deployed INDEPENDENTLY. Update one service without touching the rest.
Scalability	Scales at the service level, but shared infrastructure (ESB) can limit scalability.	Scales at the individual microservice level. Only scale the services that need it.
Technology Stack	Often uses standardized protocols (SOAP, WSDL, XML) for interoperability across different vendors and platforms.	Freedom to use different technologies per service (polyglot). Each team picks the best tool for their service.
Governance	Centralized governance — a central team manages the service directory, standards, and the ESB.	Decentralized governance — each team owns their microservice end-to-end.
Best For	Integrating EXISTING LEGACY SYSTEMS across an enterprise. Large organizations with complex, established IT landscapes.	Building NEW APPLICATIONS from scratch. Agile teams that need to deploy frequently and independently.
Main Risk	The ESB can become a bottleneck and a single point of failure. Over time, managing many services can become complex.	Operational complexity — managing hundreds of small services, distributed tracing, and service-to-service failures requires strong DevOps maturity.
Example	A bank uses SOA to connect its core banking system, CRM, loan processing system, and mobile banking app through an ESB — all discovered via a central service registry.	An e-commerce platform breaks its app into: a "Product Catalog" service, a "Shopping Cart" service, a "Payment" service, and a "Notifications" service — each deployed and scaled independently.

6.3 The Key Differences — In Plain Words

Think of it this way:

- **SOA is about integration** — connecting EXISTING systems together that were not originally built to work together. It uses a centralized bus (ESB) as the common language between systems.
- **Microservices is about application design** — building a NEW application from the start as a collection of small, independent services rather than one big block (monolith).

> **COMMON CONFUSION TO AVOID:** Students often think SOA and Microservices are the same because both use "services." They are NOT. SOA is an INTEGRATION

> STRATEGY for the enterprise. Microservices is an APPLICATION ARCHITECTURE STYLE. SOA asks: "How do we connect all these existing systems?" Microservices asks: "How do we build this new application in a flexible, deployable way?"

6.4 Can They Coexist?

Yes. In practice, many large organizations use BOTH:

- They use **SOA** to integrate their legacy enterprise systems (the old ERP, the old CRM, the old banking system) through an ESB
- They use **Microservices** to build NEW capabilities and digital products (the new mobile app, the new customer portal) in an agile, independent way
- The new microservices then communicate with the legacy SOA layer through APIs or adapters

> **EXAMPLE:** A large retail company has a 15-year-old ERP system and a 10-year-old CRM. They use SOA (with an ESB) to integrate these legacy systems. When they build a new mobile loyalty app, they build it as microservices — each running independently. The microservices call the SOA services through the ESB when they need legacy data.

6.5 SOA vs. Microservices — When to Choose Which

Situation	Choose SOA	Choose Microservices
You are dealing with...	Existing legacy systems that need to talk to each other	Building a brand new application from scratch
Your main goal is...	Enterprise-wide interoperability and integration	Agility, independent deployments, fast delivery
Your team is...	A centralized IT team managing enterprise standards	Multiple autonomous DevOps teams, each owning a service
Your data model is...	Shared across systems — a single source of truth is important	Each service owns its own data — independence is valued
Your system scale is...	Large enterprise with many heterogeneous systems	A single application that needs to scale rapidly and independently
Real-world scenario	Bank connecting core banking, CRM, and loan systems	Netflix streaming — each feature (search, recommendations, playback) is a separate microservice

> **FINAL MEMORY HOOK — SOA vs. Microservices:** "SOA = LEGO bricks in a shared toy box (centralized, enterprise-wide). Microservices = Each kid has their own set of bricks, builds independently, and passes pieces over the fence only when needed (decentralized, autonomous)."

MASTER SUMMARY — One-Page Cheat Sheet

Key Formulas to Remember

> **Architecture** = Blueprint + Rules for Evolution

> **Enterprise** = Any org with a mission and an output

> **EA** = Master plan to align IT with business goals (Business + Data + Application + Technology)

> **TOGAF** = The framework (recipe). ADM = The 9-step method. ArchiMate = The drawing language.

> **System Integration** = Making separately built systems work as one unified whole

> **SOA** = Enterprise-wide reusable services + ESB (for connecting legacy systems). Microservices = Small, independent, self-contained app services (for building new apps fast).

The 6 Integration Scenarios at a Glance

#	Scenario	The Problem It Solves	Key Word
1	Information Portal	I need to SEE data from many systems in one place	AGGREGATE / VIEW
2	Data Replication	I need the same data in multiple systems, kept in sync	COPY / SYNC
3	Shared Business Function	Multiple systems use the same data/logic — eliminate duplication	CENTRALIZE
4	Service-Oriented Architecture	Enterprise-wide reusable services, formal contracts, ESB connectivity	SERVICES + ESB
5	Distributed Business Process Management	A process spans many systems and needs a coordinator	ORCHESTRATE
6	Business-to-Business (B2B)	I need to connect to EXTERNAL organizations	EXTERNAL BRIDGE

SOA vs. Microservices — 5-Second Version

	SOA	Microservices
Purpose	Integrate existing enterprise systems	Build new applications in small parts
Communication	Central ESB bus	Direct lightweight APIs
Data	Shared data model	Each service owns its own data
Deployment	Coordinated releases	Each service deploys independently
Best for	Legacy integration, large enterprise	New apps, agile teams